Федеральное государственное автономное образовательное учреждение
высшего образования «Самарский национальный исследовательский
университет имени академика С.П. Королева»

*Кафедра технической кибернетики*

Отчет по лабораторной работе №3

**«Потоковая обработка в Apache Flink»**

Выполнил:
студент группы 6403-010302D
Сулимов Дмитрий Николаевич

Самара 2026

## ЗАДАНИЕ

Выполнить следующие задания из набора заданий репозитория https://github.com/ververica/flink-training-exercises:

- RideCleansingExercise
- RidesAndFaresExercise
- HourlyTipsExerxise
- ExpiringStateExercise

Решения могут быть выполнены на двух языках: Java или Scala. Каждому заданию соответствует .java или .scala файл с шаблоном решения и файл с тестом решения. Тесты расположены в папке test.

Для выполнения заданий вам потребуется датасет с данными о поездках такси в Нью-Йорке https://github.com/apache/flink-training/blob/master/README.md#using-the-taxi-data-streams. Файлы nycTaxiFares.gz и nycTaxiRides.gz вы можете найти в папке data https://gitlab.com/ssau.tk.courses/big_data/-/tree/master/data.

## ХОД ВЫПОЛНЕНИЯ ЛАБОРАТОРНОЙ РАБОТЫ

1. Метод filter выполняет проверку географических координат начала (startLon, startLat) и завершения (endLon, endLat) каждой поездки. С помощью вспомогательного класса GeoUtils.isInNYC проверяется вхождение обеих точек маршрута в границы Нью-Йорка. В результирующий поток попадают только те записи, которые начинаются и заканчиваются в пределах город. Код RideCleansingExercise представлен в приложении А. На рисунке 1 изображены результаты выполнения тестов для RideCleansingExercise.

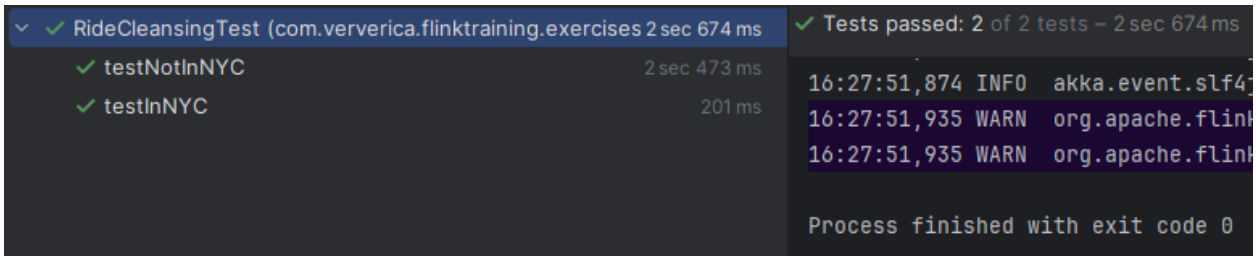


Рисунок 1 — Результаты тестов RideCleansingTest

2. Код реализует логику объединения двух независимых потоков данных в реальном времени. Основная задача состоит в том, чтобы сопоставить информацию о поездке с данными о её оплате по общему идентификатору. Поскольку события поступают в систему с задержкой или в разном порядке, в коде задействован механизм временного хранения — состояние. При получении данных из одного потока проверяется, не поступила ли информация для этой же поездки из другого потока. Если парная запись найдена, формируется итоговый результат, а временная память очищается. Если же пара еще не получена, текущие данные сохраняются в состояние до момента появления недостающей части. Код RidesAndFaresExercise представлен в приложении Б. На рисунке 2 изображены результаты выполнения тестов для RidesAndFaresExercise.

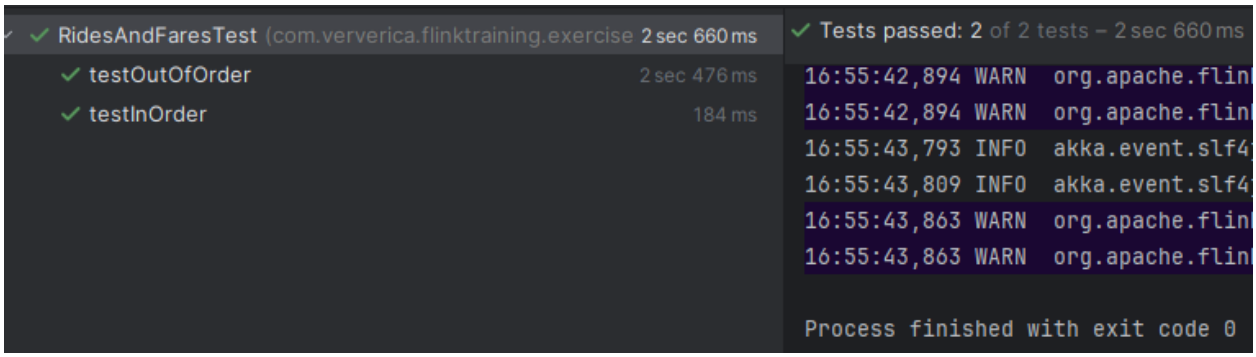


Рисунок 2 — Результаты тестов RidesAndFaresTest

3. На первом этапе поток данных группируется по идентификатору водителя (driverId), и с помощью часового окна вычисляется промежуточная сумма чаевых для каждого сотрудника. Полученные результаты направляются во второе временное окно той же длительности, но сгруппированное по метке времени окончания периода. Второе окно аккумулирует итоговые суммы всех водителей за конкретный час и с помощью оператора maxBy определяет абсолютного лидера. Использование окон на обоих этапах гарантирует, что система выдаст ровно один финальный результат для каждого часа, исключая избыточные промежуточные обновления при асинхронном поступлении данных. Код HourlyTipsExercise представлен в приложении В. На рисунке 3 изображены результаты выполнения тестов для HourlyTipsExercise.

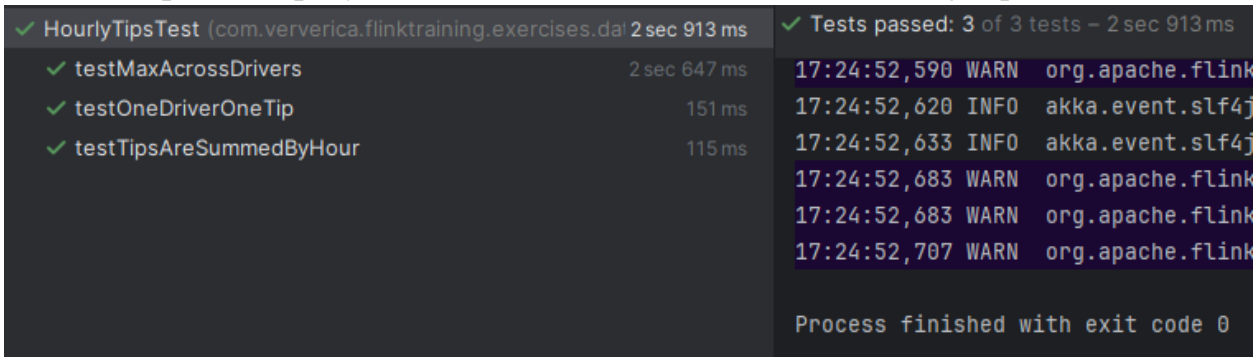


Рисунок 3 — Результаты тестов HourlyTipsTest

4. Реализован механизм обогащения данных с контролируемым временем жизни состояния с использованием функции KeyedCoProcessFunction для сопоставления поездок и оплат с обработкой исключительных ситуаций, когда одно из событий отсутствует в потоке. При поступлении элемента, не имеющего пары в состоянии, он сохраняется в ValueState, и для него регистрируется таймер на момент времени, соответствующий временной метке события. Если парное событие приходит до срабатывания таймера, пара объединяется, выводится в основной поток, а таймер удаляется. Если же таймер срабатывает, это означает, что пара не была найдена в течение допустимого времени. В этом случае событие направляется в соответствующий боковой поток для логирования ошибок, а состояние принудительно очищается. Код ExpiringStateExercise представлен в приложении Г. На рисунке 4 изображены результаты выполнения тестов для ExpiringStateExercise.

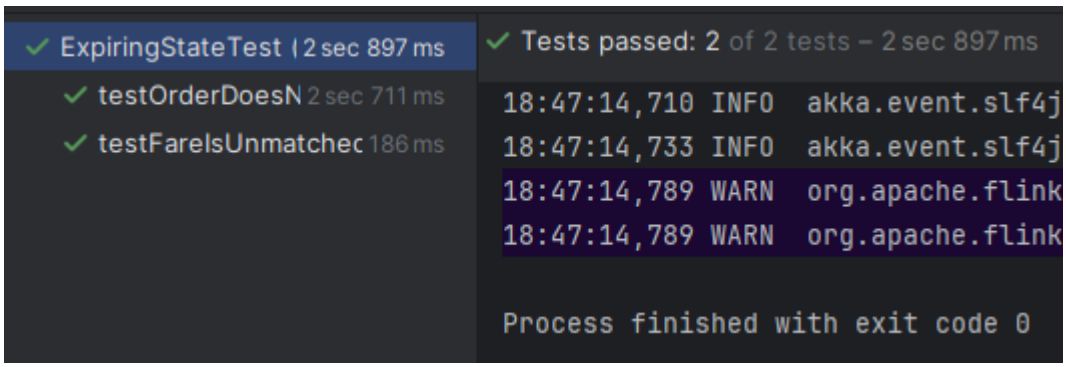


Рисунок 4 — Результаты тестов ExpiringStateTest

# ЗАКЛЮЧЕНИЕ

В ходе выполнения лабораторной работы были изучены ключевые возможности фреймворка Apache Flink для распределенной обработки потоковых данных в режиме реального времени. На примере набора данных о поездках такси в Нью-Йорке были практически освоены методы очистки, объединения асинхронных потоков и сложной агрегации данных. Использование механизмов событийного времени позволило обеспечить высокую точность расчетов, независимую от задержек при передаче информации по сети.

# ПРИЛОЖЕНИЕ А

## Код RideCleansingExercise

```
public class RideCleansingExercise extends ExerciseBase {
   public static void main(String[] args) throws Exception {

      ParameterTool params = ParameterTool.fromArgs(args);
      final String input = params.get("input", ExerciseBase.pathToRideData);

      final int maxEventDelay = 60;       // events are out of order by max 60 seconds
      final int servingSpeedFactor = 600; // events of 10 minutes are served in 1 second

      // set up streaming execution environment
      StreamExecutionEnvironment env = StreamExecutionEnvironment.getExecutionEnvironment();
      env.setParallelism(ExerciseBase.parallelism);

      // start the data generator
      DataStream<TaxiRide> rides = env.addSource(rideSourceOrTest(new TaxiRideSource(input, maxEventDelay,
servingSpeedFactor)));

      DataStream<TaxiRide> filteredRides = rides
            // filter out rides that do not start or stop in NYC
            .filter(new NYCFilter());

      // print the filtered stream
      printOrTest(filteredRides);

      // run the cleansing pipeline
      env.execute("Taxi Ride Cleansing");
   }

   private static class NYCFilter implements FilterFunction<TaxiRide> {

      @Override
      public boolean filter(TaxiRide taxiRide) throws Exception {
         return GeoUtils.isInNYC(taxiRide.startLon, taxiRide.startLat) && GeoUtils.isInNYC(taxiRide.endLon,
taxiRide.endLat);
      }
   }

}
```

# ПРИЛОЖЕНИЕ Б

## Код RidesAndFaresExercise

```
public class RidesAndFaresExercise extends ExerciseBase {
   public static void main(String[] args) throws Exception {

      ParameterTool params = ParameterTool.fromArgs(args);
      final String ridesFile = params.get("rides", pathToRideData);
      final String faresFile = params.get("fares", pathToFareData);

      final int delay = 60;              // at most 60 seconds of delay
      final int servingSpeedFactor = 1800;   // 30 minutes worth of events are served every second

      // set up streaming execution environment
      StreamExecutionEnvironment env = StreamExecutionEnvironment.getExecutionEnvironment();
      env.setParallelism(ExerciseBase.parallelism);

      DataStream<TaxiRide> rides = env
          .addSource(rideSourceOrTest(new TaxiRideSource(ridesFile, delay, servingSpeedFactor)))
          .filter((TaxiRide ride) -> ride.isStart)
          .keyBy("rideId");

      DataStream<TaxiFare> fares = env
          .addSource(fareSourceOrTest(new TaxiFareSource(faresFile, delay, servingSpeedFactor)))
          .keyBy("rideId");

      DataStream<Tuple2<TaxiRide, TaxiFare>> enrichedRides = rides
          .connect(fares)
          .flatMap(new EnrichmentFunction());

      printOrTest(enrichedRides);

      env.execute("Join Rides with Fares (java RichCoFlatMap)");
   }

   public static class EnrichmentFunction extends RichCoFlatMapFunction<TaxiRide, TaxiFare, Tuple2<TaxiRide,
TaxiFare>> {

      private ValueState<TaxiRide> rideState;
      private ValueState<TaxiFare> fareState;

      @Override
      public void open(Configuration config) throws Exception {
         ValueStateDescriptor<TaxiRide> rideDescriptor = new ValueStateDescriptor<>("saved-ride",
TaxiRide.class);
         ValueStateDescriptor<TaxiFare> fareDescriptor = new ValueStateDescriptor<>("saved-fare",
TaxiFare.class);

         rideState = getRuntimeContext().getState(rideDescriptor);
         fareState = getRuntimeContext().getState(fareDescriptor);
         rideState = getRuntimeContext().getState(rideDescriptor);
         fareState = getRuntimeContext().getState(fareDescriptor);
      }

      @Override
      public void flatMap1(TaxiRide ride, Collector<Tuple2<TaxiRide, TaxiFare>> out) throws Exception {
```

```
        TaxiFare fare = fareState.value();
        if (fare != null) {
          fareState.clear();
          out.collect(new Tuple2<>(ride, fare));
        } else {
          rideState.update(ride);
        }
      }

      @Override
      public void flatMap2(TaxiFare fare, Collector<Tuple2<TaxiRide, TaxiFare>> out) throws Exception {
        TaxiRide ride = rideState.value();
        if (ride != null) {
          rideState.clear();
          out.collect(new Tuple2<>(ride, fare));
        } else {
          fareState.update(fare);
        }
      }
    }
}
```

# ПРИЛОЖЕНИЕ В

## Код HourlyTipsExercise

```
public class HourlyTipsExercise extends ExerciseBase {

   public static void main(String[] args) throws Exception {

      // read parameters
      ParameterTool params = ParameterTool.fromArgs(args);
      final String input = params.get("input", ExerciseBase.pathToFareData);

      final int maxEventDelay = 60;       // events are out of order by max 60 seconds
      final int servingSpeedFactor = 600; // events of 10 minutes are served in 1 second

      // set up streaming execution environment
      StreamExecutionEnvironment env = StreamExecutionEnvironment.getExecutionEnvironment();
      env.setStreamTimeCharacteristic(TimeCharacteristic.EventTime);
      env.setParallelism(ExerciseBase.parallelism);

      // start the data generator
      DataStream<TaxiFare> fares = env.addSource(fareSourceOrTest(new TaxiFareSource(input, maxEventDelay,
servingSpeedFactor)));

      DataStream<Tuple3<Long, Long, Float>> hourlyTips = fares
            .keyBy(fare -> fare.driverId)
            .window(TumblingEventTimeWindows.of(Time.hours(1)))
            .process(new AddTips());

      DataStream<Tuple3<Long, Long, Float>> hourlyMax = hourlyTips
            .keyBy(t -> t.f0)
            .window(TumblingEventTimeWindows.of(Time.hours(1)))
            .maxBy(2);

      printOrTest(hourlyMax);

      // execute the transformation pipeline
      env.execute("Hourly Tips (java)");
   }

   public static class AddTips extends ProcessWindowFunction<
            TaxiFare, Tuple3<Long, Long, Float>, Long, TimeWindow> {

      @Override
      public void process(Long key, Context context, Iterable<TaxiFare> fares, Collector<Tuple3<Long, Long,
Float>> out) {
         float sumOfTips = 0F;
         for (TaxiFare f : fares) {
            sumOfTips += f.tip;
         }
         // Результат: (конец окна, ID водителя, сумма чаевых)
         out.collect(new Tuple3<>(context.window().getEnd(), key, sumOfTips));
      }
   }

}
```

# ПРИЛОЖЕНИЕ Г

## Код ExpiringStateExercise

```
public class ExpiringStateExercise extends ExerciseBase {
    static final OutputTag<TaxiRide> unmatchedRides = new OutputTag<TaxiRide>("unmatchedRides") {};
    static final OutputTag<TaxiFare> unmatchedFares = new OutputTag<TaxiFare>("unmatchedFares") {};

    public static void main(String[] args) throws Exception {

        ParameterTool params = ParameterTool.fromArgs(args);
        final String ridesFile = params.get("rides", ExerciseBase.pathToRideData);
        final String faresFile = params.get("fares", ExerciseBase.pathToFareData);

        final int maxEventDelay = 60;           // events are out of order by max 60 seconds
        final int servingSpeedFactor = 600;    // 10 minutes worth of events are served every second

        // set up streaming execution environment
        StreamExecutionEnvironment env = StreamExecutionEnvironment.getExecutionEnvironment();
        env.setStreamTimeCharacteristic(TimeCharacteristic.EventTime);
        env.setParallelism(ExerciseBase.parallelism);

        DataStream<TaxiRide> rides = env
            .addSource(rideSourceOrTest(new TaxiRideSource(ridesFile, maxEventDelay, servingSpeedFactor)))
            .filter((TaxiRide ride) -> (ride.isStart && (ride.rideId % 1000 != 0)))
            .keyBy(ride -> ride.rideId);

        DataStream<TaxiFare> fares = env
            .addSource(fareSourceOrTest(new TaxiFareSource(faresFile, maxEventDelay, servingSpeedFactor)))
            .keyBy(fare -> fare.rideId);

        SingleOutputStreamOperator processed = rides
            .connect(fares)
            .process(new EnrichmentFunction());

        printOrTest(processed.getSideOutput(unmatchedFares));

        env.execute("ExpiringStateExercise (java)");
    }

    public static class EnrichmentFunction extends KeyedCoProcessFunction<Long, TaxiRide, TaxiFare,
Tuple2<TaxiRide, TaxiFare>> {
```

```
    private ValueState<TaxiRide> rideState;
    private ValueState<TaxiFare> fareState;

    @Override
    public void open(Configuration config) throws Exception {
      rideState = getRuntimeContext().getState(new ValueStateDescriptor<>("rideState", TaxiRide.class));
      fareState = getRuntimeContext().getState(new ValueStateDescriptor<>("fareState", TaxiFare.class));
    }

    @Override
    public void onTimer(long timestamp, OnTimerContext ctx, Collector<Tuple2<TaxiRide, TaxiFare>> out)
throws Exception {
      if (fareState.value() != null) {
        ctx.output(unmatchedFares, fareState.value());
        fareState.clear();
      }
      if (rideState.value() != null) {
        ctx.output(unmatchedRides, rideState.value());
        rideState.clear();
      }
    }

    @Override
    public void processElement1(TaxiRide ride, Context context, Collector<Tuple2<TaxiRide, TaxiFare>> out)
throws Exception {
      TaxiFare fare = fareState.value();
      if (fare != null) {
        fareState.clear();
        context.timerService().deleteEventTimeTimer(fare.getEventTime());
        out.collect(new Tuple2<>(ride, fare));
      } else {
        rideState.update(ride);
        context.timerService().registerEventTimeTimer(ride.getEventTime());
      }
    }

    @Override
    public void processElement2(TaxiFare fare, Context context, Collector<Tuple2<TaxiRide, TaxiFare>> out)
throws Exception {
      TaxiRide ride = rideState.value();
      if (ride != null) {
        rideState.clear();
```

```
            context.timerService().deleteEventTimeTimer(ride.getEventTime());
            out.collect(new Tuple2<>(ride, fare));
        } else {
            fareState.update(fare);
            context.timerService().registerEventTimeTimer(fare.getEventTime());
        }
    }
  }
}
```